Phase 8: Project Demonstration — Scalability Roadmap & Future Expansion Plan

Project Name: Cab Booking (`UCab`)

Project ID: `N/A (Solo Track Submission)`

Team Member / Solo Developer: Shaik Sumiya Zainab

1. Future Scalability Architecture Plan

While Phase 1 delivers a feature-complete and robust MVP capable of supporting hundreds of concurrent simulated trips, expanding UCab into a multi-city commercial enterprise requires specific architectural upgrades across Phase 2 and Phase 3.

```
timeline
title UCab Scalability & Evolution Roadmap
Phase 1 : Core MVP (Current Delivery)
: MERN Monolith on Port 8000
: Zero-Config Dual Database Storage
: Simulated GPS Progress Bar
: QR Corporate Thermal Invoices
Phase 2 : Real-Time & Payment Webhooks (Months 1-3)
: Bi-Directional WebSocket Driver Streaming (Socket.io)
: Live Stripe & Razorpay Webhook Checkout
: Driver Partner App & Live GPS Push
Phase 3 : Distributed Microservices & AI (Months 4-6)
: Microservice Split (Auth, Fleet, Dispatch, Billing)
: Redis Caching for Sub-Millisecond Fleet Lookup
: AI/ML Surge Pricing & Demand Prediction via GeoJSON
```

2. Technical Roadmap Breakdown

Phase 2: Real-Time Telemetry & Production Payment Gateway (Next 90 Days)

1. WebSocket / `Socket.io` Integration: Replace the stateful polling/simulation tracking bar in `UserHome.jsx` with bi-directional WebSocket channels (`ws://localhost:8000`). Drivers will push live GPS coordinates every 3 seconds, rendering an animated marker on a Mapbox/Google Maps canvas.

2. Stripe & Razorpay Webhook Verification: Connect `bookingController.js` directly to Stripe payment intents, verifying card verification cryptograms and webhook events (`payment_intent.succeeded`) before marking trips as `Accepted`.

Phase 3: Distributed Microservices & AI/ML Forecasting (Months 4 to 6)

1. Microservice Decomposition: Split the single Express API server (`server.js`) into four isolated Dockerized microservices:

- `ucab-auth-service`: Handles JWT issuing, OAuth, and Bcrypt validation.
- `ucab-fleet-service`: Manages vehicle inventory, image attachments, and categorization.
- `ucab-dispatch-service`: Handles trip matching and driver assignment.
- `ucab-billing-service`: Generates QR invoices and manages tax compliance.

2. Redis In-Memory Caching: Introduce Redis (`Redis Cloud / ElastiCache`) in front of `/api/cars` to cache available fleet inventory, reducing database query latency from `12ms` down to `< 1ms` during peak city rush hours.

3. AI/ML Demand Surge Forecasting: Integrate a Python machine learning service (trained on historical trip coordinates) to predict surge demand across specific urban geofences, adjusting per-KM rates dynamically to incentivize drivers to move into high-demand zones.